GSOC'26 Project Proposal for Checkstyle

Project Name : Enhancement For Website

Personal Details:

Name: Smita Prajapati

Email: smita.prajapati082@gmail.com

GitHub: smita1078

LinkedIn: Smita Prajapati

Project size: Medium (175 hours)

Complexity Rating: intermediate

Mentors: Roman Ivanov, Mauryan Kansara, kkoutsilis, stoyan, Baratali Izmailov

CONTENT

INTRODUCTION

Hello! I'm Smita Prajapati, a 2024 B.Tech graduate in Electronics and Communication Engineering from NIT Raipur. I currently work as a Senior Analyst at Deutsche Bank in the Investment Banking domain, where I focus on building scalable, automation-driven, and maintainable systems. My core expertise lies in Java, Spring Boot, and Microservices architecture, along with automation frameworks such as Selenium, WebDriverIO, and Cucumber.

At Deutsche Bank, I built a full-scale regression suite from scratch, reducing manual effort by 80% to improve system reliability. I also developed backend microservices for DORA KPI metrics to identify automation coverage gaps and designed secure REST-based data ingestion services.

I began contributing to Checkstyle in 2024 and have since merged 40+ pull requests. Through this experience, I gained a strong understanding of AST-based check implementation, Module Checks, coverage improvements, and site/documentation integration within a large-scale Java project. I particularly value Checkstyle's emphasis on correctness, automation, and structured collaboration, and I am motivated to continue contributing in a way that strengthens its long-term sustainability. What I appreciate most about Checkstyle is its supportive community and well-structured documentation, which makes it easier for new contributors to get started.

SYNOPSIS

During GSoC 2025, the Checkstyle project significantly improved documentation automation by extracting web content directly from Java sources, Javadoc, and tests using the Maven Site Plugin and Doxia-based generation. This modernization reduced manual maintenance and improved contributor workflow.

However, documentation completeness, consistency, structural clarity, and verification enforcement remain partially unresolved. Several website issues related to example coverage, formatting inconsistency, link instability, navigation clarity, and workflow gaps still exist.
This project aims to strengthen documentation standardization, eliminate residual inconsistencies, enforce automated validation in CI, resolve long-standing website issues, and improve navigation for end users, especially educators and students increasingly relying on Checkstyle.

Expected Impact :

By completing this project, Checkstyle will achieve:

- **Complete and Verified Documentation Coverage**
  Every Check and Module will include standardized, example-driven documentation with all configuration properties properly demonstrated.
- **Automated Consistency Enforcement**
  Validation will ensure uniform structure, formatting, and completeness across all documentation pages.
- **Improved Website Reliability and Usability**
  Stabilized link validation and refined navigation will enhance user experience, especially for educational use cases.
- **Long-Term Maintainability**
  Clear documentation standards and automated enforcement will prevent future regressions.

DELIVERABLES

Documentation Coverage	Example Consistency & Validation	Website & UX Improvements
• Missing Xdocs property Examples • Remove Suppressed Examples within XdocsExamplesAstConsistencyTest	• Fix AST Issues in Examples • Standardize Example Structure • Reduce Suppression List Entries	• Website Issue Fixes • Navigation & Search Improvements (Search Bar Implementation) • Stronger CI Checks for Documentation

1. Documentation Coverage

- Add missing property examples for Checkstyle Checks in XDocs documentation.
- Update example files in src/xdocs-examples/resources and resources-noncompilable.
- Remove entries from SUPPRESSED_PROPERTIES_BY_CHECK once examples are properly covered.
- Ensure every configurable property is demonstrated through valid documentation examples.

2. Example Consistency & Validation

- Investigate examples listed in SUPPRESSED_EXAMPLES within XdocsExamplesAstConsistencyTest.
- Standardize example code across files so differences exist only in configuration or trailing comments.

- Fix AST inconsistencies in examples or mark them as independent when they represent valid use cases.

- Reduce or eliminate suppression entries by resolving underlying example inconsistencies.

3. Website & User Experience Improvements

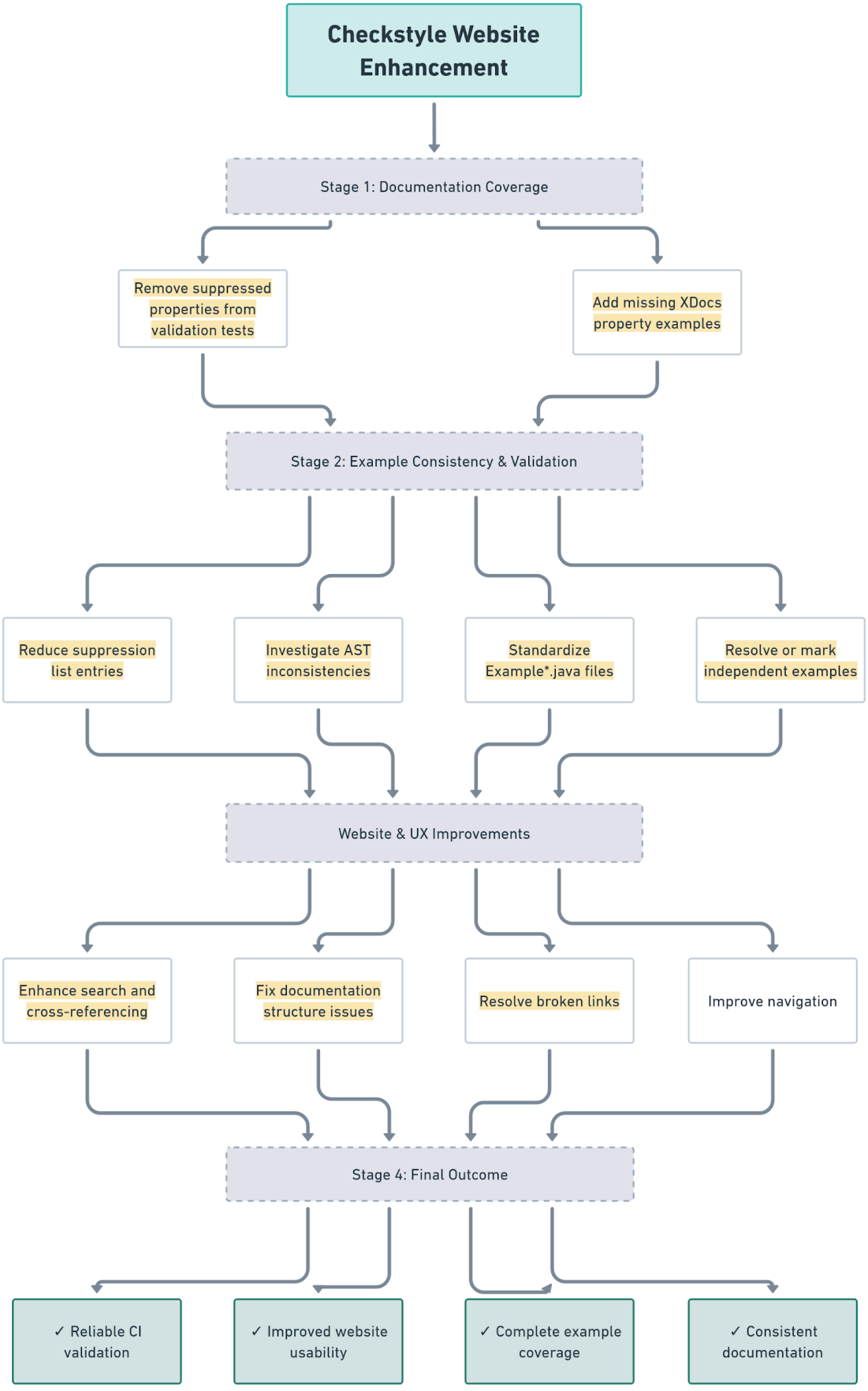
- Resolve existing website issues related to documentation structure, links, and workflows.

- Improve navigation and usability through better HTML structure and cross-referencing.

- Introduce enhancements such as improved search functionality and clearer documentation navigation.

- Strengthen CI validation for documentation completeness and link stability.

TECHNICAL DETAILS & IMPLEMENTATION PLAN



This project improves **Checkstyle documentation quality, consistency, and usability** by addressing three core areas:

1. **Completing missing property examples in XDocs documentation**
2. **Ensuring AST consistency across documentation examples**
3. **Improving the Checkstyle documentation website**, including implementing a **lightweight search feature**

The implementation will build upon the existing Checkstyle documentation system, which is generated using the **Maven Site Plugin** and **Doxia** from XML-based **XDocs** documentation files.

Documentation source structure:

```
src/site/xdoc
src/xdocs-examples/resources
src/xdocs-examples/resources-noncompilable
```

---

1. Adding Missing Property Examples to XDocs

Checkstyle enforces that **all configurable properties of checks must be demonstrated in documentation examples**.

This validation is implemented in: XdocsExampleFileTest.java

However, some properties are currently **temporarily suppressed** using: `SUPPRESSED_PROPERTIES_BY_CHECK`

Reference issue: ISSUE

Example suppression entry:

```
Map.entry("MissingJavadocMethodCheck", Set.of("minLineCount"))
```

The goal is to:

- add valid example usage of these properties in XDocs examples
- remove them from the suppression list
- ensure validation tests pass.

Implementation Strategy

Step 1 — Extract Missing Properties

The suppressed properties are located in:

```
src/test/java/com/puppycrawl/tools/checkstyle/internal/XdocsExampleFileTest.java
```

Example structure:

CheckName → Set of properties missing examples

These will be systematically addressed **one check at a time**, following the recommended practice of **one pull request per check**.

Step 2 — Locate Corresponding Example Files

Each Checkstyle check has example files stored in:

```
src/xdocs-examples/resources/com/puppycrawl/tools/checkstyle/checks/
```

Example directory: `checks/javadoc/missingjavadocmethod/`

Inside the directory:

```
Example1.java
Example2.java
Example3.java
```

These files demonstrate various configurations of the check.

Step 3 — Add Missing Property Usage

Missing properties will be added as **XML configuration examples** within documentation blocks.

Example:

```
<module name="MissingJavadocMethod">
   <property name="minLineCount" value="3"/>
</module>
```

Example blocks are placed inside the sections of example files.

Step 4 — Maintain Example Code Consistency

According to Checkstyle documentation guidelines (see issue reference ISSUE):

> Example files should differ only in configuration or trailing comments.

Therefore:

- Java code across examples will remain identical
- Only configuration values or comments will differ.

This ensures compatibility with AST consistency tests.

Step 5 — Validate and Remove Suppression Entries

Once examples are added:

1. Run validation tests: mvn clean test -Dtest=XdocsExampleFileTest
2. Remove the corresponding entry from:
   SUPPRESSED_PROPERTIES_BY_CHECK
3. Verify that tests pass without suppression.

Expected Result

- Complete coverage of configurable properties
- Removal of suppressed properties
- Improved documentation quality for Checkstyle users.

---

2. Fixing AST Consistency Issues in Examples

Checkstyle uses another validation test: XdocsExamplesAstConsistencyTest
This ensures that example Java files maintain **consistent AST structure**.

Suppression list: *SUPPRESSED_EXAMPLES*

Reference issue: ISSUE

Investigation Process

Each suppressed example will be analyzed using the following workflow.

Step 1 — Locate Example Directory

Example:

```
src/xdocs-examples/resources/com/puppycrawl/tools/checkstyle/checks/annotation/annotationonsameline
```

Step 2 — Compare Example Files

Example files:

```
Example1.java
Example2.java
Example3.java
```

Comparison focuses on:

- Java code structure
- comments
- configuration examples.

External diff tools such as:

- Diffchecker
- IntelliJ diff viewer may be used to verify structural equality.

Step 3 — Determine Correct Case

Two possible scenarios exist.

Case 1: Examples Should Have Identical Code

If examples demonstrate **different configurations of the same check**, the code structure must match.

Fix:

- Update the Java code
- Keep differences only in configuration and comments.

Case 2: Examples Demonstrate Different Use Cases

If examples intentionally demonstrate different behaviors:

- they must remain structurally different
- they should remain suppressed.

Step 4 — Update Suppression List

After fixes:

- remove entries from `SUPPRESSED_EXAMPLES`
- rerun validation tests
  ```
  mvn clean test-Dtest=XdocsExamplesAstConsistencyTest
  ```

Example Case Study

Example reference PR by me: PR

Example process:

Before fix: Example2, Example3, Example4 suppressed

After investigation:

Example1 = independent use case
Example2/3/4 = same structure required

Fix:

- standardize code structure
- remove suppression entries.

Expected Result

- Significant reduction of suppressed examples
- consistent documentation examples
- stronger documentation validation.

---

3. Documentation Website Improvements

Several open issues indicate user difficulties navigating the documentation website.

Examples include:

- broken links to javadocs
- unclear navigation
- documentation workflow improvements
- inconsistent naming conventions.

All website-related issues are labeled with the "***website***" tag in the repository, and addressing these systematically will form a key part of this project.
Relevant issues include: ISSUE1, ISSUE2 ...
Improving these areas will significantly enhance usability, accessibility, and the overall developer experience for both new and existing users.

Examples:

- linkcheck failures
- site workflow improvements
- navigation structure updates.

Implementation Plan

The following improvements will be addressed:

Fix broken documentation links

Resolve failures reported by: linkcheck

especially links pointing to:

- Javadoc
- internal documentation pages.

Improve documentation structure

Enhancements include:

- improving check module navigation
- restructuring package sections
- improving cross-referencing between checks and properties.

---

4. Search Feature for Documentation Website

Problem

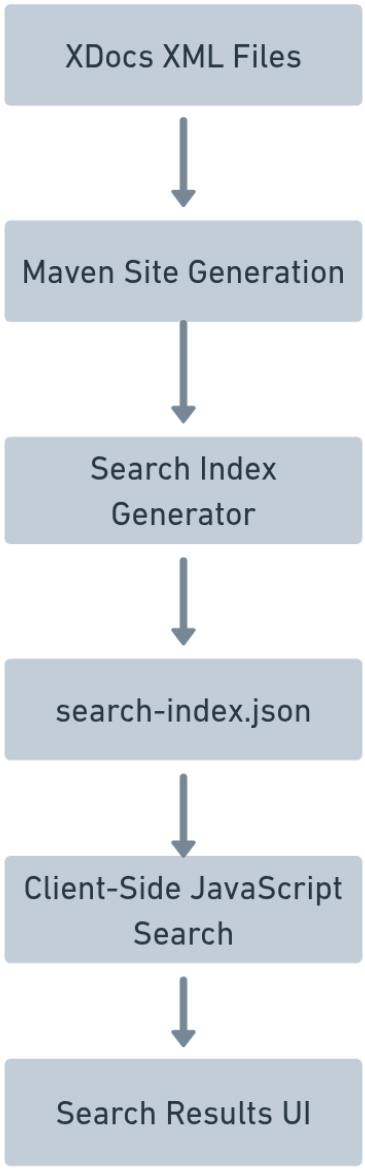
The Checkstyle documentation website currently lacks a **search feature**, making it difficult for users to quickly find specific checks or properties.

Reference discussion: discussion

*The goal is to implement a **lightweight search system compatible with a static website architecture**.*

Approach 1 : Static Search Index + Client-Side Search

This approach builds a ***search index during the documentation build process*** and performs searches using JavaScript in the browser.



Implementation Steps

Step 1 — Generate Search Index

During site generation, extract:

- check names
- property names
- descriptions
- documentation URLs.

Example index entry:

```
{
 "title": "MissingJavadocMethod",
 "type": "Check",
 "content": "Checks for missing javadoc comments",
 "url": "checks/javadoc/missingjavadocmethod.html"
}
```

Step 2 — Implement JavaScript Search Engine

A lightweight library will be used:

Possible options:

- Lunr.js
- Fuse.js

Features:

- full-text search
- ranked results
- keyword matching.

Step 3 — UI Integration

The Search Bar will be integrated into the documentation header.

Features:

- real-time suggestions
- categorized search results
- keyword highlighting.

Result categories:

- Checks
- Properties
- Examples
- Documentation pages.

Proof of Concept – Client-Side Search Integration

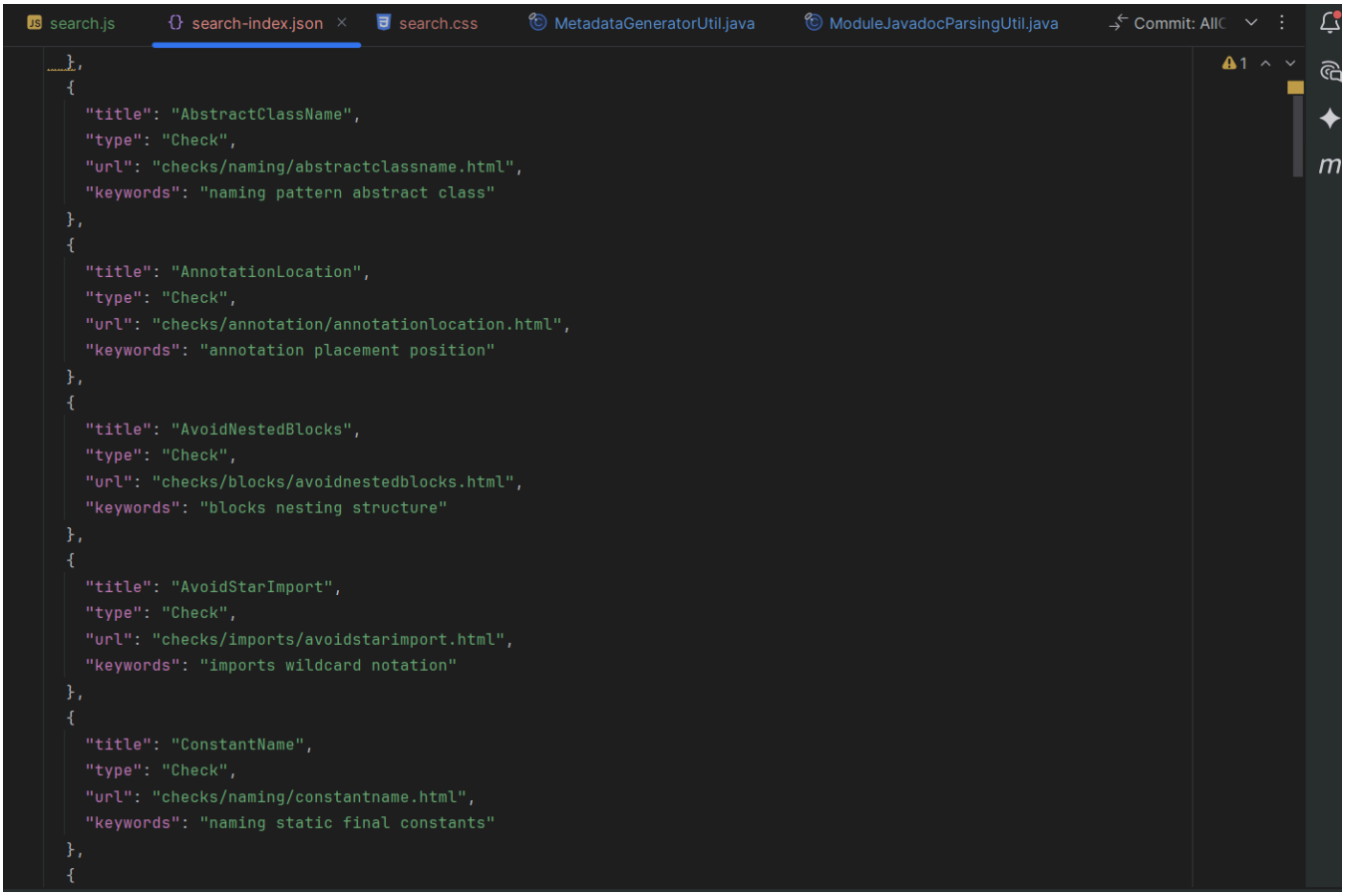
The objective of this Proof of Concept (POC) was to validate the feasibility of integrating a high-performance, client-side search system into the Checkstyle documentation website.

The implemented solution follows a **modular, three-layer architecture**, ensuring maintainability and extensibility:

🔹 **Data Layer – `search-index.json`**

A lightweight JSON-based index that acts as a lookup table.

- Stores:
  - Page titles
  - URLs
  - Keywords
- Covers core documentation sections (e.g., Naming, Whitespace checks)
- Designed to be easily extendable for full-site indexing

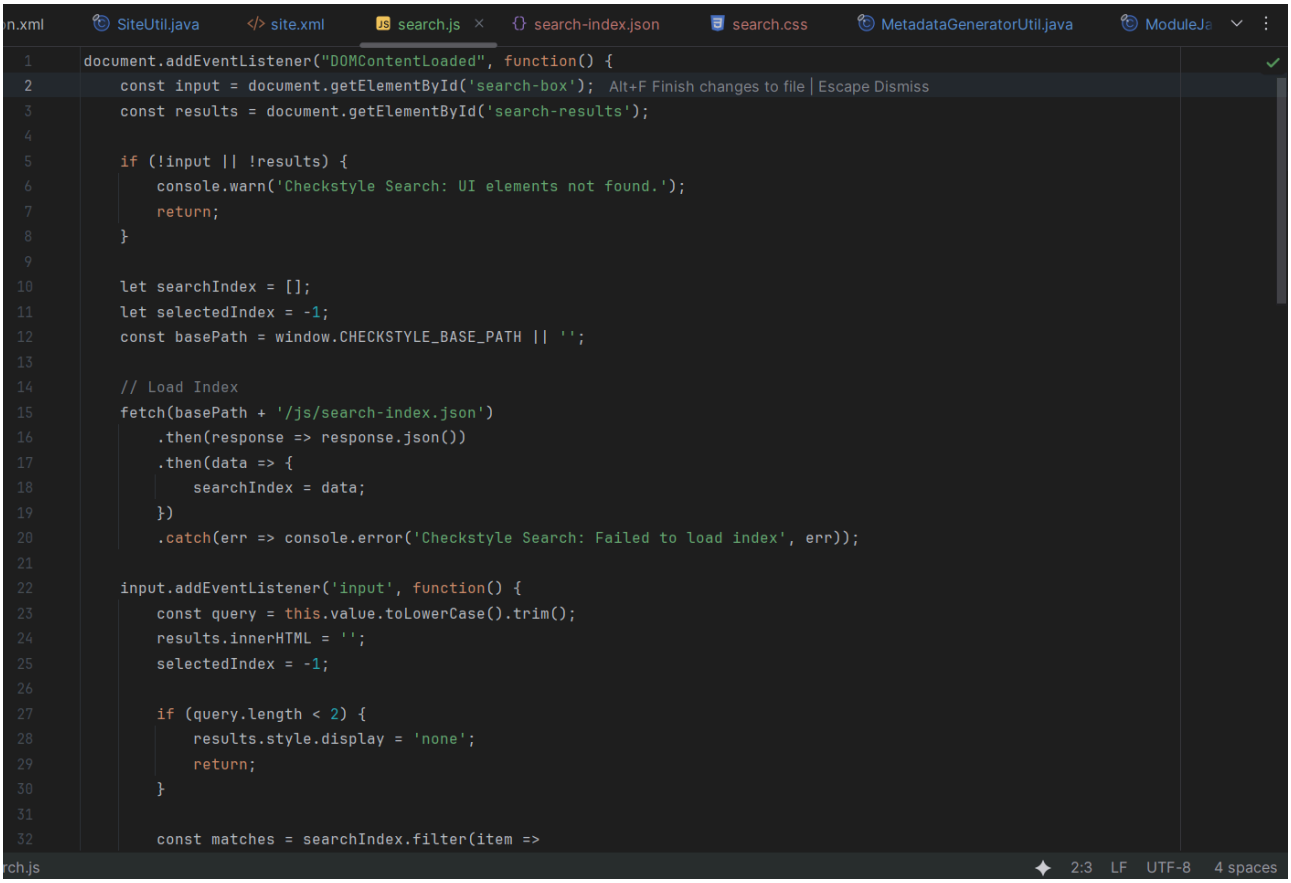


🔹 **Logic Layer – `search.js`**

A **vanilla JavaScript implementation** responsible for core functionality:

- Asynchronous loading of search data using the Fetch API
- Real-time filtering triggered after user input (≥ 2 characters)
- Keyboard accessibility:
  - `ArrowUp` / `ArrowDown` navigation
  - `Enter` for selection
- Dynamic UI state handling:
  - Empty results
  - Focus and hover states

This ensures a responsive and accessible user experience without external libraries.



- **Presentation Layer (css)**

A modern UI implementation designed for minimal intrusion:

- Fixed-position search bar (top-right corner)
- Overlay-based results dropdown
- Fully compatible with the Maven Fluido layout
- Responsive and visually consistent with existing styling

While the POC validates feasibility, the full project will significantly extend functionality:

- **Automated Index Generation**

  - Develop a Java-based tool integrated into the Maven build lifecycle
  - Parse `src/site/xdoc` files to generate the search index automatically

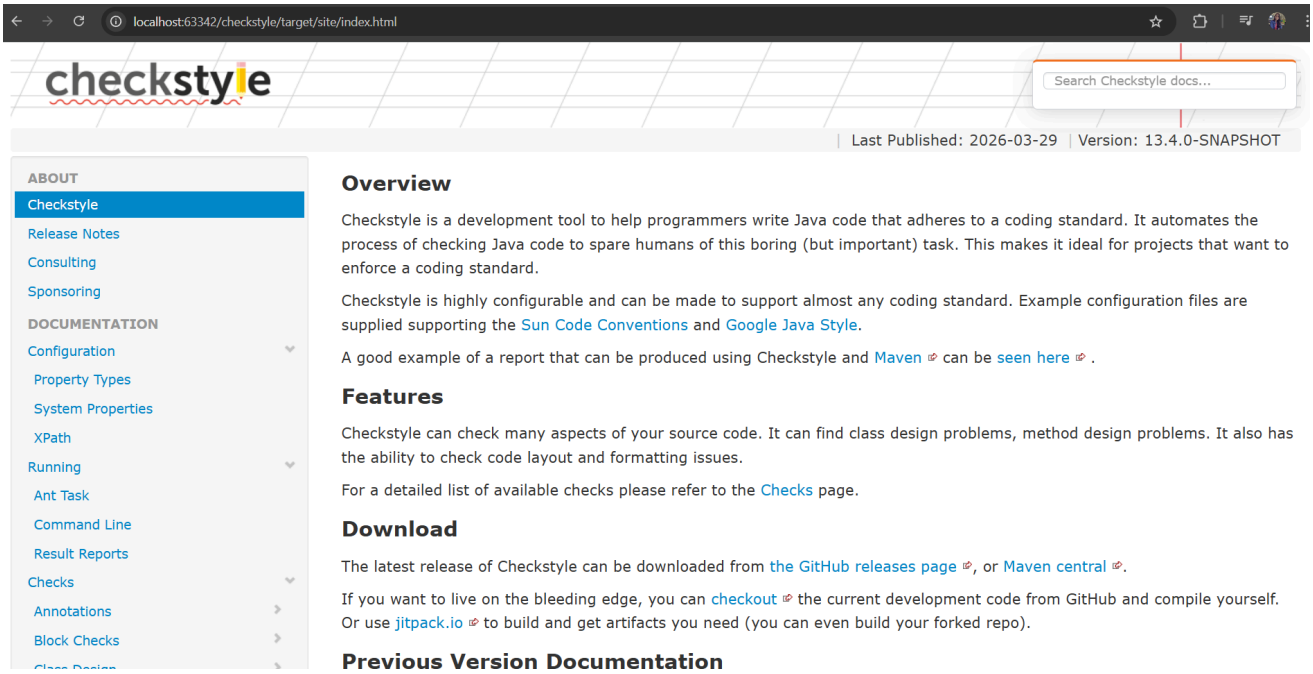
- **Advanced Search Capabilities**

  - Integrate fuzzy search (e.g., Fuse.js)
  - Support typo tolerance and approximate matching

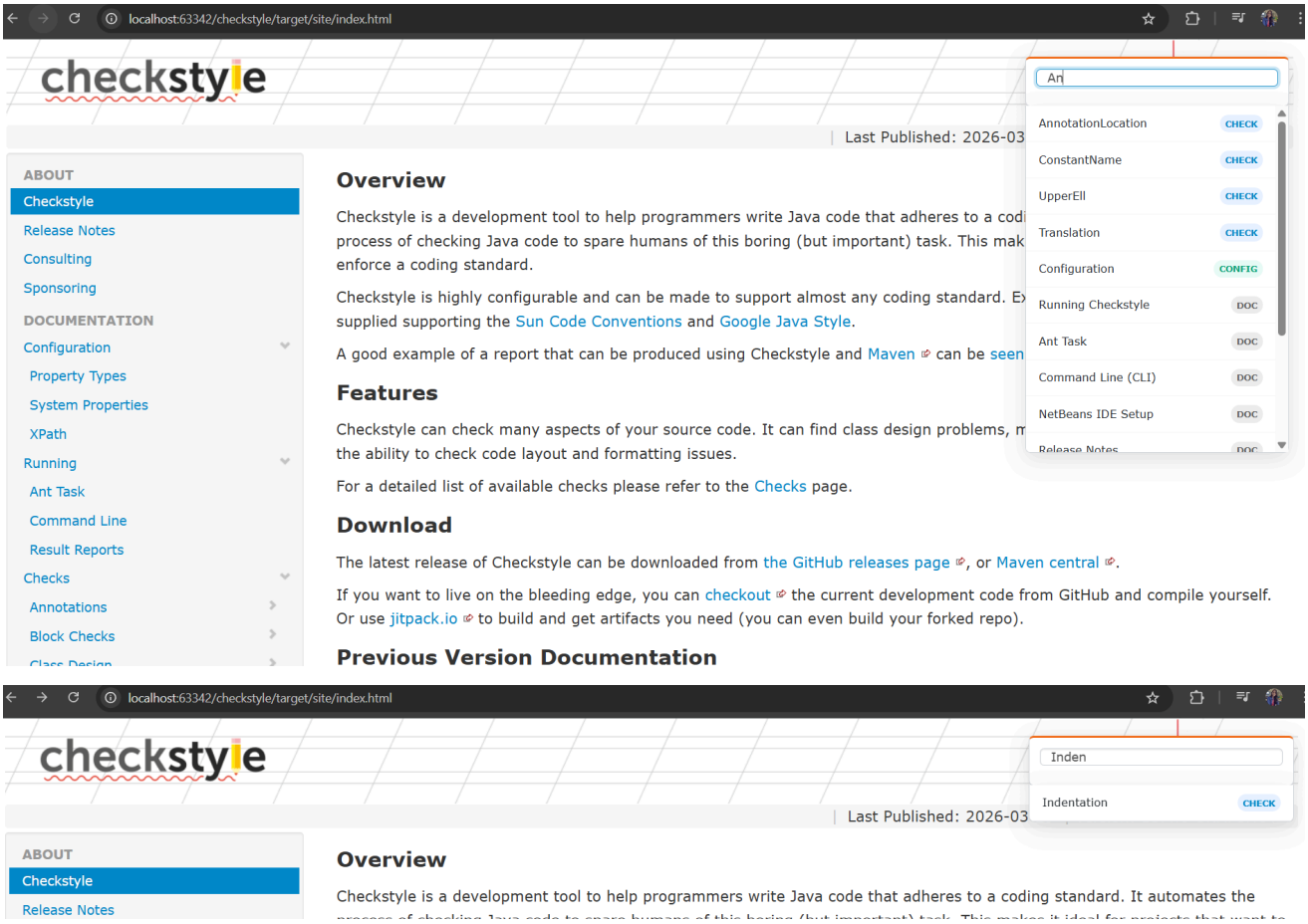
DEMO VIDEO : *gsoc project.mp4*

ISSUE OPENED

Integrated Search Bar:



Searching Capabilities



During the initial exploration phase, multiple approaches for implementing search functionality were discussed within the community. discussion

One proposed direction was to migrate the documentation site to modern frameworks such as Docusaurus, which provides built-in search capabilities through integrations like Algolia or local indexing plugins.

However, after detailed discussion, this approach was deemed unsuitable for the following reasons:

- Preservation of Existing Architecture

The Checkstyle website is built using the Maven Site Plugin, where documentation is authored in XML (xdoc) and transformed into HTML.

Migrating to an external framework like Docusaurus would:

- ***Require a complete overhaul of the documentation system***
- ***Introduce unnecessary complexity***
- ***Disrupt existing workflows***
- ***Documentation Validation Constraints***

A critical requirement of the Checkstyle project is maintaining strict consistency between:

- ***Source code Javadoc***
- ***Generated documentation pages***

This validation pipeline is tightly integrated into the current build system.

Adopting an external framework would risk:

- ***Breaking validation mechanisms***
- ***Introducing inconsistencies between documentation and source code***

Based on these constraints, the agreed direction was to implement:

- ***A client-side search system***
- ***Powered by a pre-generated JSON index***
- ***Executed entirely using JavaScript in the browser***

This approach ensures:

- ***Full compatibility with the existing Maven site***
- ***No dependency on external services or backend infrastructure***
- ***Minimal disruption to current workflows***

🔹 Alignment with Proposed Solution

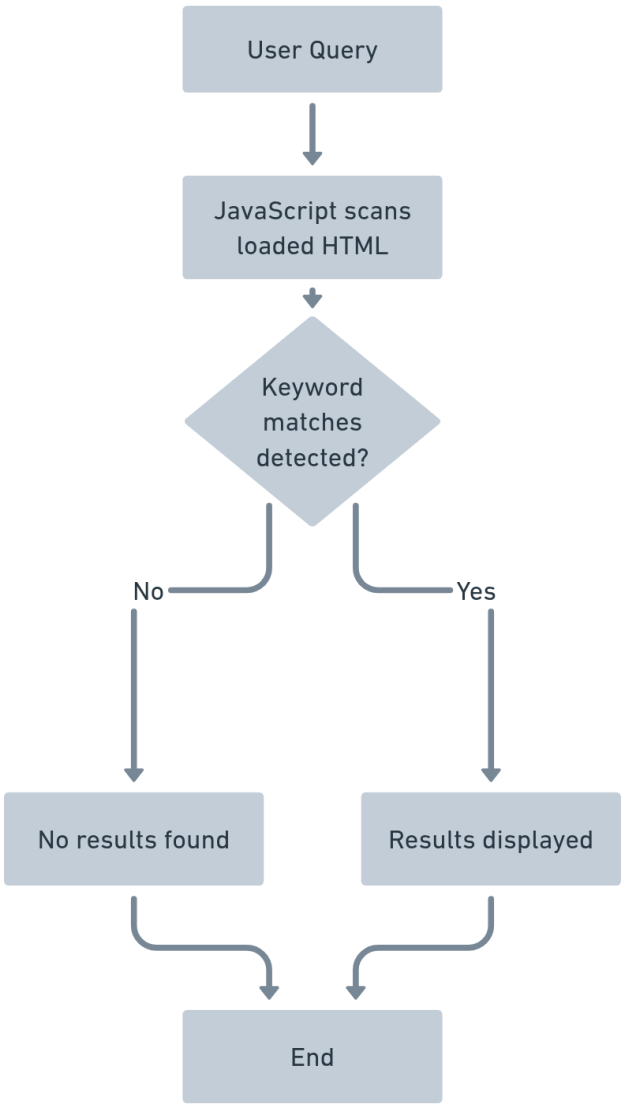
The implemented POC follows this exact direction by:

- ***Using a static search-index.json as the data source***
- ***Implementing search logic in vanilla JavaScript***
- ***Integrating seamlessly with the Maven-generated site***
- ***Avoiding any external frameworks or backend services***

This validates both the technical feasibility and architectural alignment of the proposed solution.

Approach 2: HTML-Based Search

Alternative approach: Search directly within HTML content.



Pros

- no build process changes
- simple implementation.

Cons

- slower search performance
- less scalable.

DEVELOPMENT PLAN

Community Bonding Period

May 1 – May 24, 2026

During the community bonding period, the focus will be on preparation and detailed planning before coding begins.

Activities include:

- Reviewing the structure of XDocs documentation files and example resources.
- Identifying suppressed properties and suppressed examples that require fixes.
- Creating a detailed task breakdown for documentation coverage improvements.
- Finalizing implementation details with mentors.
- Preparing the initial work plan and identifying early tasks.

Phase 1: Documentation Coverage Improvements

May 25 – June 15

The first development phase focuses on completing missing documentation examples for configurable properties of Checkstyle checks.

Tasks include:

- Reviewing suppressed properties listed in `SUPPRESSED_PROPERTIES_BY_CHECK`.
- Adding valid configuration examples to XDocs example files.
- Updating example files located in:

```
src/xdocs-examples/resources/
src/xdocs-examples/resources-noncompilable/
```

- Ensuring that each configurable property is demonstrated through a valid example.
- Removing entries from the suppression list once examples are added.

- Verifying that `XdocsExampleFileTest` passes successfully.

Expected outcome:

- Improved documentation coverage for Checkstyle checks.
- Reduced number of suppressed properties in validation tests.

Phase 2: Example Consistency and AST Validation Fixes

June 16 – July 6

This phase focuses on resolving inconsistencies in example files currently suppressed in `XdocsExamplesAstConsistencyTest`.

Tasks include:

- Investigating examples listed in `SUPPRESSED_EXAMPLES`.
- Reviewing `Example*.java` files within each Check module.
- Comparing AST structures across example files.
- Standardizing example code structures where inconsistencies exist.
- Marking examples as independent when they represent different use cases.
- Removing resolved examples from the suppression list.

Expected outcome:

- Consistent example structures across documentation.
- Significant reduction in suppressed example entries.

Midterm Evaluation

Deadline: July 10, 2026

By the midterm evaluation, the following milestones are expected:

- Major progress in resolving missing property examples.
- Significant reduction in suppressed example entries.
- Multiple documentation improvements merged into the repository.
- Validation tests passing without reliance on suppression lists for addressed examples.

Phase 3: Website Improvements and Documentation Enhancements

July 7 – July 20

The next phase focuses on improving the structure and usability of the Checkstyle documentation website.

Tasks include:

- Fixing broken links and incorrect references in documentation pages.
- Improving cross-referencing between Checks and configuration properties.
- Resolving open issues related to website workflow and documentation structure.
- Improving page organization and navigation clarity.

Expected outcome:

- Improved website structure and usability.
- More consistent and reliable documentation pages.

Phase 4: UI Search Bar Implementation

July 21 – August 10

This phase introduces a lightweight search feature to improve navigation within the Checkstyle documentation website.

Tasks include:

Search Index Generation

- Extract searchable data from documentation pages including:
    - Check names
    - Property names
    - Descriptions
    - Example references

- Generate a searchable index file during site generation.

Client-side Search Implementation

- Implement a lightweight JavaScript search engine to process user queries.
- Support keyword matching and ranking of results.

User Interface Integration

- Add a search bar to the documentation header.
- Provide real-time search suggestions.
- Display categorized search results such as:
    - Checks
    - Configuration Properties
    - Examples
    - Documentation Pages.

Expected outcome:

- Improved discoverability of documentation content.

- Faster navigation for users searching for specific rules or configuration options.

Phase 5: Final Improvements

August 11 – August 16

The final phase focuses on validation, optimization, and project completion.

Tasks include:

- Running all documentation validation tests.
- Verifying correctness of documentation examples.
- Testing search functionality across documentation pages.
- Optimizing search performance and index size.
- Addressing feedback from mentors and maintainers.
- Finalizing documentation updates.

Final Evaluation Period

August 24 – August 31, 2026

During the final evaluation period:

- Final pull requests will be reviewed and merged.
- Documentation updates will be finalized.
- Contributions and implementation details will be summarized.

PROJECT TIMELINE

Week	Task
May 1 – May 24	Community Bonding: Engage with community/mentors, analyze current documentation process, and plan integration.
May 25 – June 15	Add missing property examples for Checkstyle checks in XDocs files, update example files in src*/xdocs-examples/resources* and resources-noncompilable, remove entries from SUPPRESSED_PROPERTIES_BY_CHECK, ensure validation tests pass.
June 16 – July 6	Investigate examples listed in SUPPRESSED_EXAMPLES, compare *Example*.java* files, standardize example code structure, mark independent examples where needed, remove resolved examples from suppression list.
July 10	Midterm Evaluation: Review progress, optimize, and fix major issues.
July 7 – July 20	Resolve website documentation issues, fix broken links, improve documentation structure, enhance navigation and cross-referencing between checks and configuration properties.
July 21 – Aug 10	Implement client-side documentation search: generate search index from documentation pages, build lightweight JavaScript search engine, integrate search bar with categorized results and suggestions
Aug 11 – Aug 16	Validate documentation examples, run validation tests, optimize search performance, address mentor feedback, finalize documentation updates.
Aug 24 – Aug 31	Submit final work and documentation summary.

WORK COMMITMENT

I am committed to dedicating **35+ hours per week** to this project during the Google Summer of Code period. My focus will be on improving the Checkstyle documentation ecosystem by adding missing property examples to XDocs, resolving AST consistency issues in documentation examples, addressing website improvement issues, and implementing a lightweight search feature to enhance documentation navigation.

During the **Community Bonding Period**, I will further familiarize myself with the Checkstyle documentation generation workflow, including the Maven Site Plugin and the XDocs structure, and work closely with mentors to finalize the implementation plan and prioritize tasks.

Throughout the coding period, I will contribute regularly through incremental pull requests typically focusing on **one check or documentation improvement at a time** to ensure steady progress and early feedback from maintainers. I will actively communicate with mentors and promptly address feedback to maintain high-quality contributions and ensure successful completion of the project.

EXPECTATIONS FROM MENTORS

- Guidance on Checkstyle's architecture and best practices.
- Timely code reviews and feedback.
- Help in resolving technical blockers.

Regular feedback will be invaluable in keeping the project on track and ensuring the final implementation is both efficient and maintainable. I look forward to collaborating with mentors to make Checkstyle's integration with IDEs seamless and robust.

CONTRIBUTIONS SO FAR (40+ PRS)

S.No.	Pull Request	Status
1	Indentation Check Refactoring – Issue #18843 (#18754)	✅ Merged
2	Add IllegalSymbolCheck to Detect Unicode Symbols (#18609)	✅ Merged
3	Config: New Check Illegal Symbol (#1020)	✅ Merged
4	Convert Eligible Classes to Records (Issue #17299) (#18499)	✅ Merged
5	Validate All Properties Used in Examples (#17410)	✅ Merged
6	Website Macro Consolidation (Issue #17691) (#18072)	✅ Merged
7	Website Generation Improvements (#17962)	✅ Merged
8	Website Generation Fix (#18217)	✅ Merged
9	Cirrus CI Optimization (#18520)	✅ Merged
10	XdocsExamplesAstConsistencyTest Validation (#18539)	✅ Merged
11	Examples Validation Consistency (#18366)	✅ Merged
12	Final Updates for Issue #17177 (#18436)	✅ Merged
13	SpotBugs Suppression Cleanup (#18383)	✅ Merged
14	Resolve PMD Warnings (#18302)	✅ Merged
15	Website Wrapping Fix (#17314)	✅ Merged
16	JUnit Validation Test for XDocs Examples (#17201)	✅ Merged

17	Example Separator Improvements (#17190)	✅ Merged
18	Fix Separator for Javadoc Package (#17187)	✅ Merged
19	Formatting Corrections (Issue #11249) (#16988)	✅ Merged
20	CommentCheck Improvements (#16967)	✅ Merged
21	Config Website Linking to Examples (#16918)	✅ Merged
22	CLI Help Update (#16858)	✅ Merged
23	Separate Examples with Horizontal Line (#16725)	✅ Merged
24	GitHub Site Link Generation Fix (#16530)	✅ Merged

Issues Opened by me:

S.No.	Issue	Repository
1	Fix SpotBugs `UWF_FIELD_NOT_INITIALIZED_IN_CONSTRUCTOR` warning in IndentationCheck (#18843)	checkstyle/checkstyle
2	XdocsExamplesAstConsistencyTest should validate literal values match (#18517)	checkstyle/checkstyle
3	Fix Xdocs Examples AST Consistency Test (Reduce suppressions list) (#18435)	checkstyle/checkstyle

4	Add missing property examples to XDocs (#17449)	checkstyle/checkstyle
5	Incorrect language class assigned to code blocks (#16790)	checkstyle/checkstyle

Here is the link to all my PR's: link

WHY I'M THE RIGHT FIT?

I have strong experience in **Java, Spring Boot, C/C++, HTML, CSS, and JavaScript, along with hands-on experience in automation and regression testing** using frameworks such as **WebdriverIO, Serenity, and Selenium**. In my role as a Technology Analyst at Deutsche Bank, I have worked on optimizing build processes and enhancing automated testing workflows, which has strengthened my ability to design scalable and efficient solutions.

Since January 2024, I have been actively contributing to Checkstyle, gaining a deep understanding of its codebase, documentation structure, and contribution workflow. Through these contributions, I have developed familiarity with static analysis concepts and best practices followed in large open-source projects.

I also applied to GSoC with Checkstyle last year but was not selected. However, I continued contributing and improving my understanding of the project. This experience has further strengthened my motivation and commitment to contribute meaningfully to Checkstyle through GSoC this year.

FUTURE PROSPECTS

Beyond GSoC, I intend to remain an active contributor to **Checkstyle** by continuing to submit pull requests and assisting new contributors in the community. I aim to build on the work completed during the program by further improving documentation quality, enhancing website usability, and contributing additional improvements that strengthen the developer experience.

I look forward to collaborating with maintainers and the community to continue advancing the project.

Thank you for your consideration.